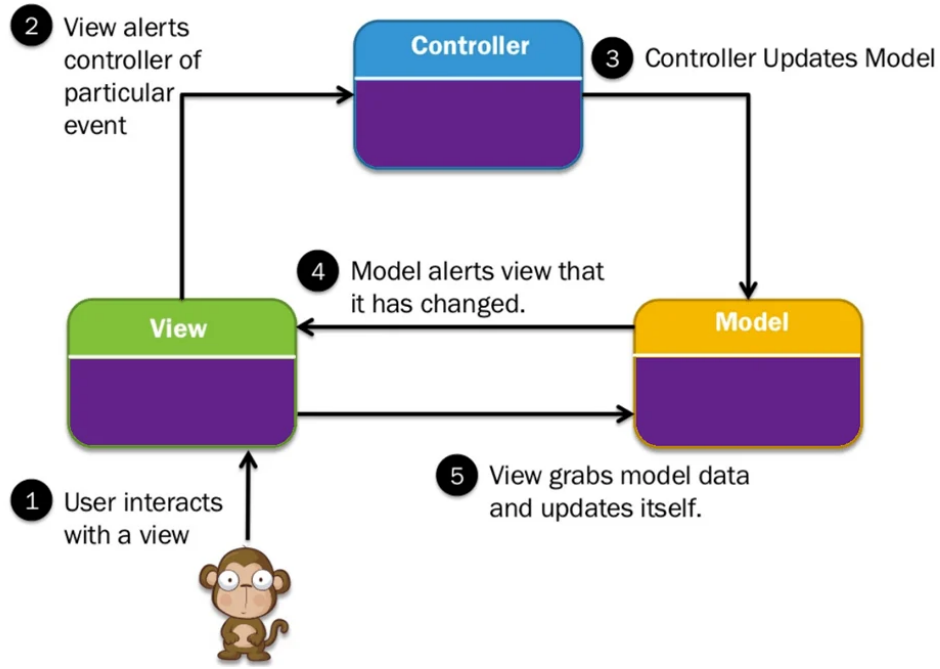
1. Introduction

This report analyzes the architectural decisions and challenges encountered while building a Java desktop banking application using RMI (Remote Method Invocation) for distributed communication, MVVM (Model-View-ViewModel) for the user interface, and JDBC for database connectivity. The application allowed administrators and clients to perform banking operations across a network with a responsive JavaFX interface.

---

2. Architecture: Why MVVM Over MVC

2.1 The Core Problem with MVC in Distributed Systems



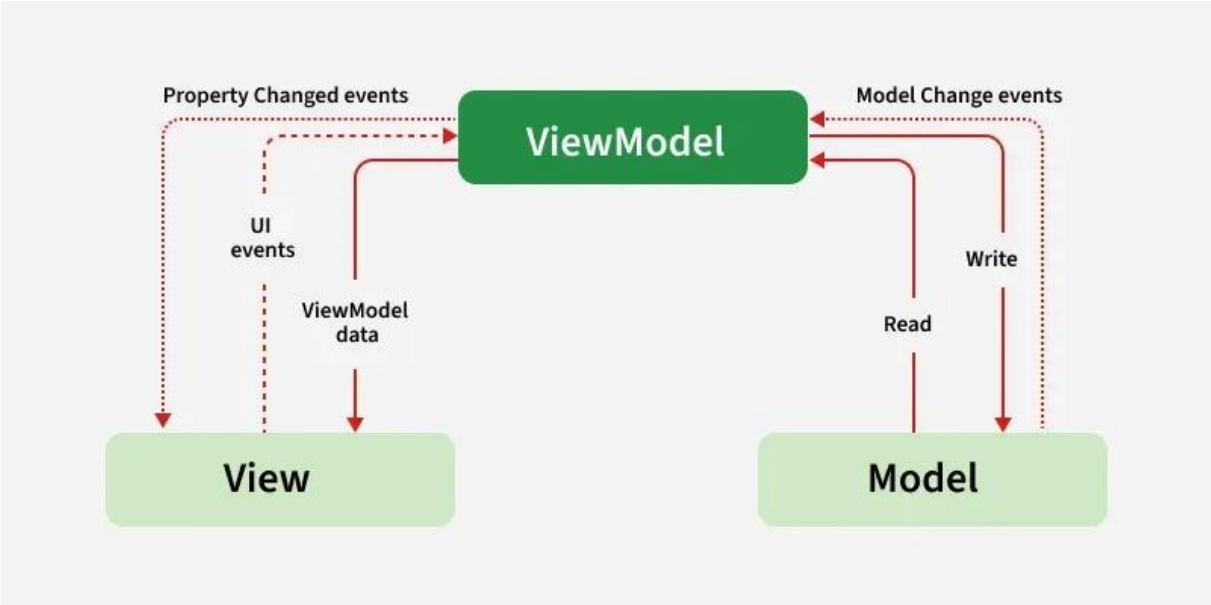
Traditional MVC (Model-View-Controller) presents two fundamental problems when combined with RMI :

Problem	Description
Manual Synchronization	Every model change requires a manual view update. With network latency, missing a single update leaves the UI stale.
Tight Coupling	MVC controllers directly reference UI widgets (@FXML private Label balanceLabel), making them untestable without starting the JavaFX runtime.

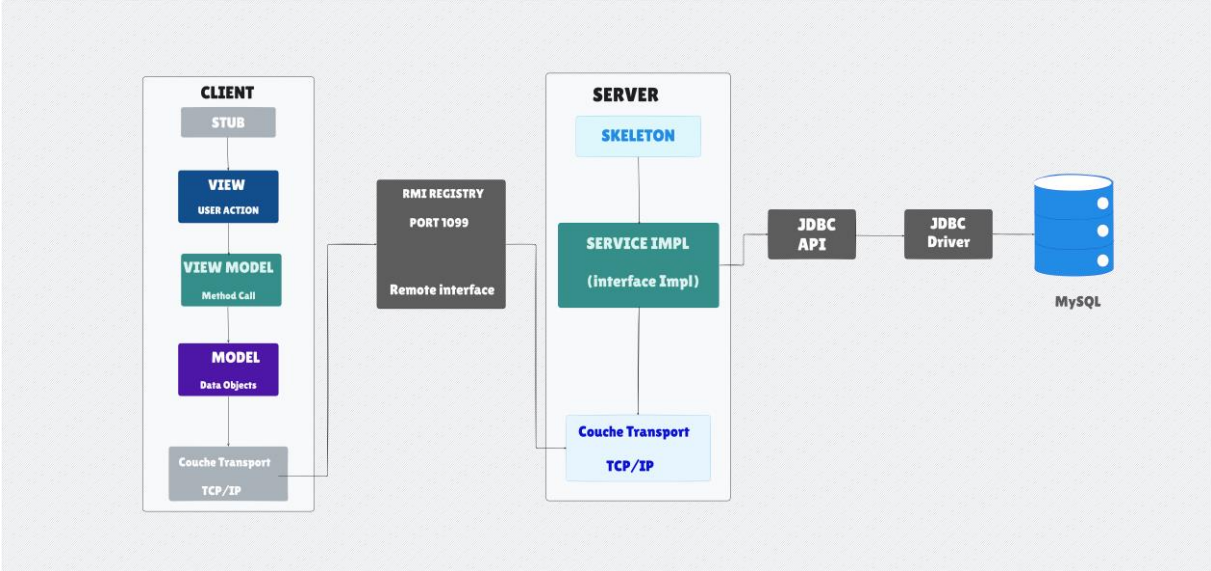
2.2 How MVVM Solves These Problems

MVVM introduces a **ViewModel** layer that acts as a "translator" between the View and the remote Model. Unlike MVC controllers, the ViewModel:

- Exposes **bindable properties** (e.g., StringProperty, DoubleProperty)
- Has **zero dependencies** on UI widgets
- Can be unit-tested without starting the JavaFX runtime



2.3 Layer Breakdown



View: Pure JavaFX FXML. Contains no business logic. Binds directly to ViewModel properties.

ViewModel: Exposes StringProperty, DoubleProperty, ObservableList. Handles RMI calls via ServiceLocator. Contains validation logic. Fully testable without UI.

Model (Server-side): RMI remote implementation (BanqueServiceImpl) + DAO layer (DbManager) + Database.

---

3. Difficulties Encountered

3.1 Network Latency Freezing the UI

The Problem: RMI calls made directly from the JavaFX Application Thread block the UI until the remote operation completes . When a user clicked "Withdraw," the entire interface froze for 500-2000ms depending on network conditions.

Why This Happens: JavaFX uses a single Event Dispatch Thread (EDT) to process all UI events. Blocking this thread with a network call prevents repaints, button clicks, and any user interaction.

3.2 Serialization Issues

The Problem: RMI requires all objects passed between client and server to implement Serializable . Forgetting to declare serialVersionUID led to InvalidClassException when server and client had different compilation versions.

Specific Failure: Adding a new field to Compte or Transaction without managing serialVersionUID broke all existing connections.

3.3 RMI Callback Complexity

The Problem: When the server-side data changed (e.g., another client performed a transfer), the ViewModel had no way of knowing. This required implementing a callback mechanism where the client registers itself with the server for notifications.

Implementation Challenge: Callbacks require the client object to be exported via UnicastRemoteObject.exportObject() before being passed to the server . This is non-trivial to implement correctly.

3.4 RemoteException Handling

The Problem: Every RMI call can throw RemoteException due to network failures, server downtime, or serialization errors. Wrapping every call in try-catch blocks cluttered the ViewModel code.

---

4. Solutions Applied

4.1 Background Threading with SwingWorker (JavaFX Alternative)

Since the application used JavaFX, Task<V> and Service<V> were used instead of SwingWorker. The pattern remains the same :

```
// In ViewModel
public void withdraw(String accountNumber, double amount) {
  Task<Boolean> task = new Task<Boolean>() {
    @Override
    protected Boolean call() throws Exception {
      return remoteService.retirer(accountNumber, amount);
    }
  };

  task.setOnSucceeded(event -> {
    boolean success = task.getValue();
    updateMessageProperty(success ? "Withdrawal successful" : "Insufficient funds");
```

```
});
new Thread(task).start()
```

Key Principle: The UI remains responsive because the RMI call executes on a background thread. The ViewModel updates its bindable properties on success/failure, and the View updates automatically via binding .

4.2 SerialVersionUID Management

Solution Applied:

- Every remote-accessible class (Compte, Utilisateur, Transaction) declared explicit serialVersionUID
- Used private static final long serialVersionUID = [unique_value]L;
- Documented that any field changes require incrementing this value

```
public class Compte implements Serializable {
  private static final long serialVersionUID = 5676201359327767535L;
  // fields and methods
}
```

4.3 Callback Implementation

Solution Applied: The ViewModel implements a callback interface and registers with the server.

```
// Client-side callback interface
public interface ClientCallback extends Remote {
  void onBalanceChanged(String accountNumber, double newBalance) throws RemoteException;
}


// ViewModel registers itself after login
public class ClientDashVM implements ClientCallback {
  public void registerWithServer() {
    ClientCallback stub = (ClientCallback) UnicastRemoteObject.exportObject(this, 0);
    remoteService.registerCallback(SessionManager.getUsername(), stub);
  }



  @Override
```

```
    public void onBalanceChanged(String accountNumber, double newBalance) {
        Platform.runLater(() -> { balanceProperty.set(newBalance);       });     }    }
```

Note: Platform.runLater() is essential because the callback arrives on an RMI thread, not the JavaFX thread .

4.4 Centralized Exception Handling

Solution Applied: A wrapper class transparently handles RemoteException:

```
public class RemoteServiceWrapper {
    public static <T> T execute(RemoteCallable<T> call) {
        try {
            return call.call();
        } catch (RemoteException e) {
            handleNetworkError(e);
            return null;
        }
    }
}
```

---

5. Lessons Learned

What Worked Well

Aspect	Result
MVVM with JavaFX Properties	Automatic UI updates eliminated entire categories of bugs
RMI for simple CRUD operations	Transparent remote calls simplified development
SessionManager singleton	Clean user state management across the application

What Would Change Next Time

1. **Use JavaFX Service instead of raw Task** for better lifecycle management
2. **Implement connection pooling** for JDBC on the server side
3. **Add a timeout mechanism** for RMI calls to prevent hung threads
4. **Use DTOs (Data Transfer Objects)** instead of passing entities directly over RMI

---

6. Resources

Official Documentation

1. **Oracle RMI Tutorial** - Official guide to Java Remote Method Invocation
   URL: https://docs.oracle.com/javase/tutorial/rmi/
2. **UnicastRemoteObject (Oracle Docs)** - Reference for exporting remote objects in RMI
   URL:
   https://docs.oracle.com/en/java/javase/11/docs/api/java.rmi/java/rmi/server/UnicastRemoteObject.html

Community Resources

3. **Baeldung: Java RMI Tutorial** - Practical guide with working examples
   URL: https://www.baeldung.com/java-rmi

Academic Reference

4. **"Java Distributed Computing" by Jim Farley (O'Reilly)** - Comprehensive coverage of RMI patterns

---